

Практическое занятие №1

Искусственный нейрон. Активационные функции.

Искусственный нейрон как основа нейронных сетей

Что такое *биологические* нейронные сети? Если вспомнить школьную программу по биологии, то это что-то, связанное с головой, а вернее, с мозгом. Мозг есть не только у человека, но и у многих других животных. Наш мозг представляет собой сложнейшую биологическую нейронную сеть, которая принимает информацию от органов чувств (глаза, уши, нос), каким-то образом ее обрабатывает (узнает лица, распознает речь, ощущает запахи и т. д.). На основе полученных данных дает команды различным исполнительным органам (пожать руку знакомому человеку, пойти к доске по просьбе учителя, покинуть помещение при появлении неприятного запаха). Все эти действия выполняет наш мозг, у которого есть биологическая нейронная сеть, состоящая из совокупности нейронов. В головном мозге человека общая нейронная сеть состоит примерно из 90 млрд нейронов, которые соединены друг с другом миллиардными связями. Это, пожалуй, самый сложный объект, известный нам, людям. Основой этой сети является нейрон. Упрощенное строение биологического нейрона показано на рис. 3.1.

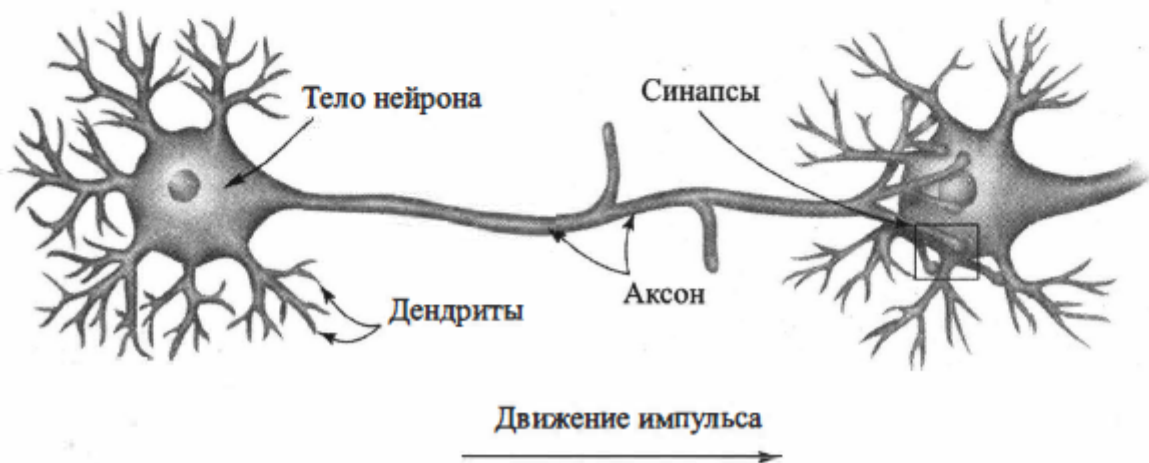


Рис. 3.1. Упрощенное строение биологического нейрона

Биологический нейрон — это нервная клетка, которая состоит из тела, дендритов и аксона. Тело соединено со множеством коротких и толстых отростков, которые называются *дендритами*. Это входные отростки — они принимают импульсы с тех нейронов, которые находятся в сети раньше, чем этот нейрон. Имеется один очень длинный и тонкий отросток, который называется *аксоном*. Это выходной отросток, по которому нейрон передает электрохимический импульс следующим нейронам в сети. Через множество дендритов нейрон получает входные сигналы, в теле нейрона они обрабатываются, а через единственный аксон выходной импульс от нейрона передается дальше. Окончание аксона имеет разветвления, через которые выходной сигнал может быть передан нескольким следующим нейронам. Реальный биологический нейрон является достаточно сложной системой. Во многом это объясняется тем, что нейрон, помимо обработки сигнала (основное его назначение), вынужден еще выполнять множество других функций, поддерживающих его жизнь. Более того, сам механизм передачи сигнала от нейрона к нейрону тоже очень сложный с биологической и химической точек зрения.

Достаточно упрощенная схема искусственного нейрона с двумя входами и одним выходом представлена на рис. 3.2.

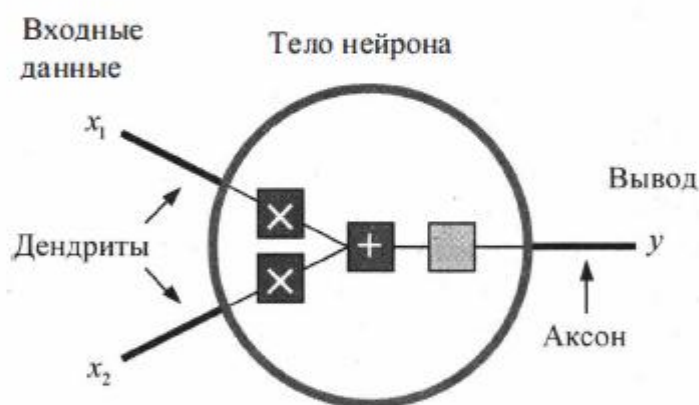


Рис. 3.2. Упрощенная схема искусственного нейрона

Здесь тело нейрона представлено в виде "черного ящика", который принимает входные данные (x_1 , x_2), выполняет с ними некоторые операции, а затем выводит результат — y .

Как нейроны взаимодействуют между собой в нейронной сети? Дело в том, что аксон каждого нейрона на своем конце имеет большое количество так называемых синапсов. *Синапс* — это место соединения выходного аксона одного нейрона с входными дендритами другого нейрона (см. рис. 3.1).

Через синапс передается нервный импульс. А сам нервный импульс формируется в теле нейрона, и фактически тело нейрона выступает сумматором, который принимает все входные импульсы, взвешивает их, передает в некоторую активирующую функцию и принимает решение, запускать импульс дальше по своему аксону или нет. Решение принимается очень просто — если суммарные входные импульсы превышают некий заданный порог, то выходной импульс запускается.

Для реализации искусственного нейрона нужно построить его упрощенную математическую модель, в которой реализованы его базовые функции без учета функций его жизнеобеспечения. В 1943 году первую модель искусственного нейрона и основанную на нем модель нейронной сети предложили американские ученые — нейрофизиолог Уоррен Мак-Каллок и математик Уолтер Питтс. Математическая модель искусственного нейрона представлена на рис. 3.3.

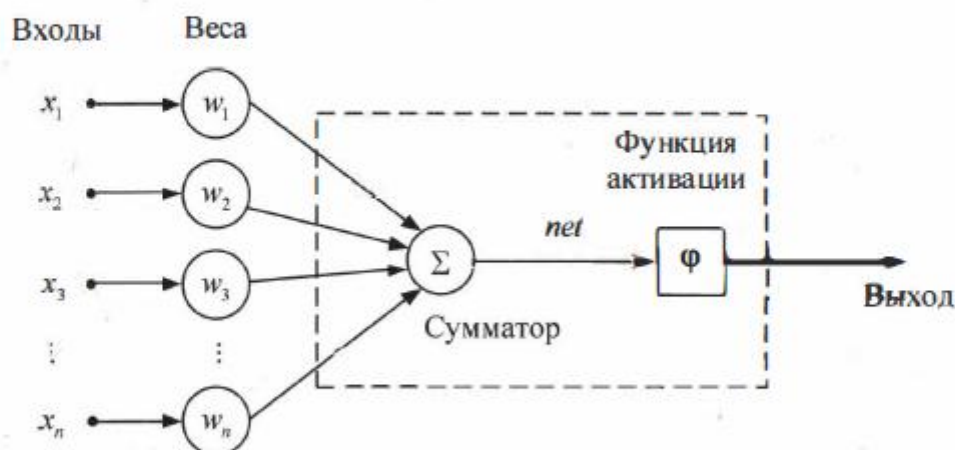


Рис. 3.3. Математическая модель искусственного нейрона

Данная модель довольно простая. На вход математического нейрона поступает некоторое количество входных параметров $x_1, x_2, x_3, \dots, x_n$, т. е. дендритов, каждый входной параметр имеет свой вес — $w_1, w_2, w_3, \dots, w_n$. В теле искусственного нейрона имеется *сумматор*, где каждый входной сигнал умножается на некоторый действительный весовой коэффициент и формируется итоговая сумма. Полученное значение передается в *функцию активации*. На выходе осуществляется проверка значения функции активации. Если ее значение выше некоторого порога, то на выход из тела нейрона передается значение единица. В этом случае нейрон активируется и в аксон передается соответствующий сигнал. Если же итоговое значение в функции активации ниже порога, то на выход из тела нейрона подается значение, равное нулю, и нейрон считается неактивированным.

Давайте в упрощенном виде рассмотрим, как работает математическая модель искусственного нейрона. У каждого нейрона, в том числе и искусственного, должны быть какие-то входы, через которые он принимает сигнал — X . Для входных сигналов вводится понятие весов — W , на которые умножаются эти сигналы. В теле искусственного нейрона поступившие на входы сигналы умножаются на их веса. Сигнал первого входа x_1 умножается на соответствующий этому входу вес w_1 , x_2 на w_2 , и так до последнего n -го входа. Затем в сумматоре эти произведения суммируются. В итоге мы получаем сумму произведений значений входных сигналов на их веса S :

$$S = x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n.$$

Роль сумматора в данном случае очевидна: он преобразует все входные сигналы (которых может быть много) в одно число — взвешенную сумму, которая характе-

ризует поступивший на нейрон сигнал в целом. Итак, на вход искусственного нейрона было подано множество входных сигналов X с их весами W , а в сумматоре мы получили единственное число — S . Реализуем сумматор в виде следующего программного кода (листинг 3.1).

Листинг 3.1

```
# Модуль Neuron
import numpy as np

# Создание класса "Нейрон"
class Neuron:
    def __init__(self, w):
        self.w = w

    def y(self, x):
        # Сумматор
        s = np.dot(self.w, x) # Суммируем входы
        return s              # функция активации

Xi = np.array([2, 3]) # Задание значений входам
Wi = np.array([1, 1]) # Веса входных сенсоров
n = Neuron(Wi)         # Создание объекта из класса Neuron
print('S1= ', n.y(Xi)) # Обращение к нейрону
Xi = np.array([5, 6]) # Веса входных сенсоров
print('S2= ', n.y(Xi)) # Обращение к нейрону
```

Здесь мы создали класс `Neuron`, в теле которого подсчитывается сумма произведений входного параметра на его вес. Если мы запустим данный программный код, то получим следующий ответ:

```
S1=5
S2=11
```

 myCompiler

English Recent Login Sign up

Enter a title...

Python

Run Save

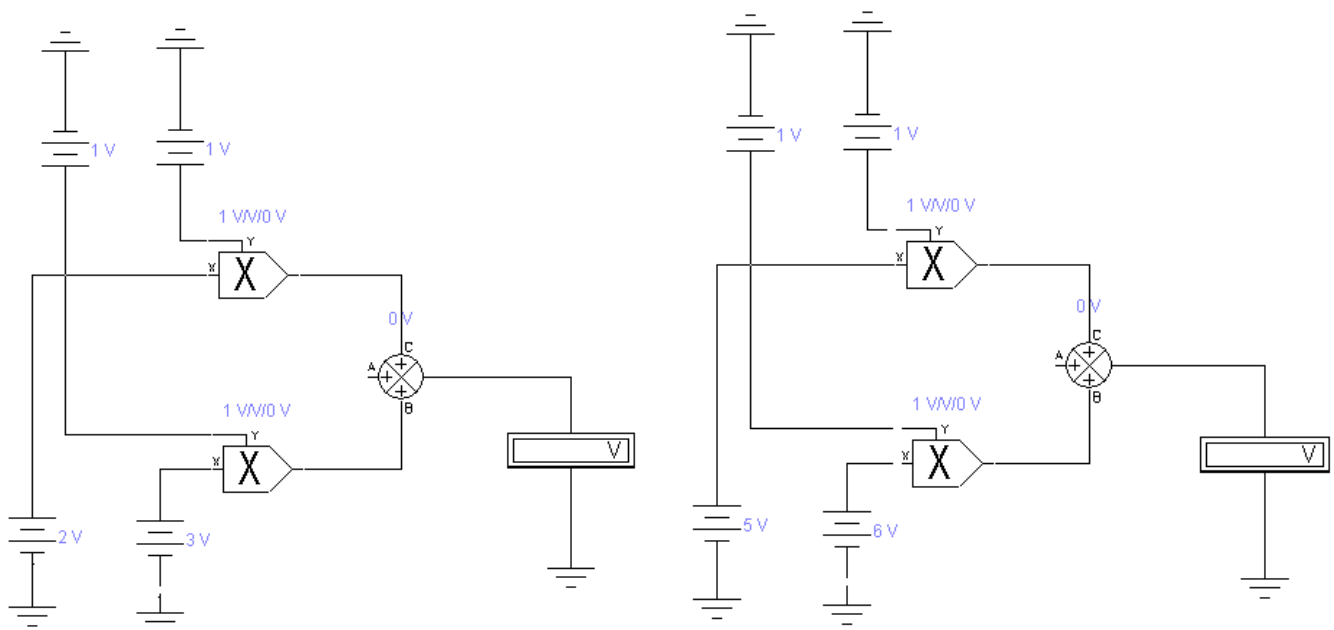
```
1 # Модуль Neuron
2 import numpy as np
3 # Создание класса "Нейрон"
4 class Neuron:
5     def __init__(self, w):
6         self.w = w
7     def y(self, x):
8         # Сумматор
9         s = np.dot(self.w, x) # Суммируем входы
10        return s              # функция активации
11
12 Xi = np.array([2, 3]) # Задание значений входам
13 Wi = np.array([1, 1]) # Веса входных сенсоров
14 n = Neuron(Wi)         # Создание объекта из класса Neuron
15 print ('S1= ', n.y(Xi)) # Обращение к нейрону
16 Xi = np.array([5, 6]) # Веса входных сенсоров
17 print ('S2= ', n.y(Xi)) # Обращение к нейрону
```

Program input

Output

S1= 5
S2= 11

[Execution complete with exit code 0]



Теперь перейдем к следующему компоненту искусственного нейрона — функции активации. Для понимания принципа работы данного компонента рассмотрим простой пример. Допустим, у нас есть один искусственный нейрон, роль которого — определить, идти на рыбалку или нет. Это типичная задача, в которой нужно проанализировать сочетание множества факторов и на основе этого анализа принять итоговое решение. Для простоты рассмотрим всего четыре фактора, влияющих на активность рыб, а значит, на успех вернуться с рыбалки с уловом. На входы в нейрон мы подадим следующие исходные данные:

- ☐ x_1 — скорость ветра;
- ☐ x_2 — атмосферное давление;
- ☐ x_3 — яркость солнца;
- ☐ x_4 — перепад температуры воды.

Рассмотрим влияние этих факторов на клев рыбы.

В зависимости от скорости ветра во многом меняются поведение рыбы в водоеме и ее интерес к приманкам. Небольшой по силе ветер всегда создает на поверхности воды рябь. Из-за нее рыба не видит четко, что находится над водой, а потому не пугается даже заметных и непривычных для водоема вещей. В такой ситуации даже такие сигналы опасности, как сторонний шум, яркий костюм рыбака, толстая или яркая леска, рыба может проигнорировать и будет активно брать наживку. Однако когда начинается сильный ветер, предчувствуя возможную бурю или шторм, рыба перестает клевать и уходит на глубину, где ей безопасно. Таким образом, при слабом ветре рыба хорошо клюет, а при сильном клев плохой.

Рыба меняет жизненную активность, скорее, не при разном значении атмосферного давления, а при его резких скачках. Наихудшими условиями для рыбалки считаются перепады давления, а также низкое давление. Хотя не все виды рыб одинаково реагируют на подобные изменения. Повышенное давление положительно сказывается на "мелочи", которая в поисках пищи перемещается в верхние слои воды и начинает активно клевать. При пониженном давлении она менее активна. Однако при пониженном давлении активизируются хищники. Мелкая рыба становится вялой, поэтому хищники тратят меньше сил и энергии в поисках пропитания. При пониженном давлении не стоит рассчитывать на клев мелкой рыбы, но зато можно поймать крупного хищника. Предположим, что мы берем с собой спиннинг и собираемся ловить щуку или судака. Таким образом, при низком давлении клев данных видов рыб будет хорошим, а при высоком давлении — плохим.

Яркое солнце не способствует хорошему улову. Поэтому если выходить на рыбалку в ясный солнечный день, то клева может не быть. Если на небе есть облака либо вы пойдете на берег ранним утром или вечером, когда солнце не такое яркое, то вероятность иметь хороший улов гораздо выше.

Рыбы чувствительны не столько к температуре воды, сколько к ее резким перепадам, реагируя даже на колебания в десятые доли градуса. Перепад в 4–5 °C (не связанный со сменой дня и ночи) негативно сказывается на активности рыб. Лучше всего рыба чувствует себя при минимальном суточном изменении температуры. Считается, что этот диапазон не должен превышать 1–2 °C. Так, если вчера дневная температура воды была 10 °C, а сегодня в это же время она составляет от 8 до 12 °C, то это нормально и можно прогнозировать пищевую активность рыбы. В противном случае при перепаде температуры более 4–5 °C активность рыбы будет снижена. Таким образом, при стабильной температуре воды клев будет хороший и можно идти на рыбалку, при резком колебании температуры воды лучше остаться дома.

Для упрощения нашей модели присвоим всем этим параметрам логические значения в виде цифр 0 или 1. Например, если ветер умеренный и мы можем идти на рыбалку, то значение входа будет 1, если на улице сильный ветер и клев маловероятен, то на входе будет 0. Соответственно если атмосферное давление низкое и щука с судаком выйдут на охоту, то на этот вход подаем 1, а если атмосферное давление высокое, то 0. Аналогично поступаем со всеми остальными параметрами: пасмур-

но — 1, солнечно — 0, температура воды стабильна — 1, сильный перепад температуры воды — 0.

Если у нейрона есть четыре входа, то должны быть и четыре весовых коэффициента. В нашем примере весовые коэффициенты можно представить как показатели важности каждого входа, влияющие на общее решение нейрона. Чем больше значение коэффициента, тем больше важность входного параметра. Веса входов распределим следующим образом:

□ $w_1 = 5$;

□ $w_2 = 4$;

□ $w_3 = 1$;

□ $w_4 = 1$.

По заданным весовым коэффициентам нетрудно заметить, что очень важную роль играют факторы скорости ветра и атмосферного давления (первые два входа). Они же и будут самыми важными при принятии нейроном решения. Вторые два фактора имеют более низкое влияние на принятие решения.

Пусть на входы нашего нейрона мы подаем следующие сигналы (ветер — умеренный, атмосферное давление — высокое, яркость солнца — пасмурно, температура воды — стабильная):

□ ветер (умеренный) $x_1 = 1$;

□ атмосферное давление (высокое) $x_2 = 0$;

□ яркость солнца (солнечно) $x_3 = 0$;

□ перепад температуры воды (нет) $x_4 = 1$.

При поступлении этой информации в сумматор он выдаст следующую итоговую сумму:

$$S = x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 = 1 \cdot 5 + 0 \cdot 4 + 0 \cdot 1 + 1 \cdot 1 = 5 + 0 + 0 + 1 = 6.$$

Проверим это с помощью нашей программы-сумматора, представленной в листинге 3.2. Для этого изменим значения соответствующих параметров в программном коде сумматора.

Листинг 3.2

```
Xi = np.array([1, 0, 0, 1]) # Задание значений входам
Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
n = Neuron(Wi)              # Создание объекта из класса Neuron
print('S= ', n.y(Xi))       # Обращение к нейрону
```

Если запустить программу на выполнение, то мы действительно получим результат $S=6$.

Enter a title...

Python



Run

Save

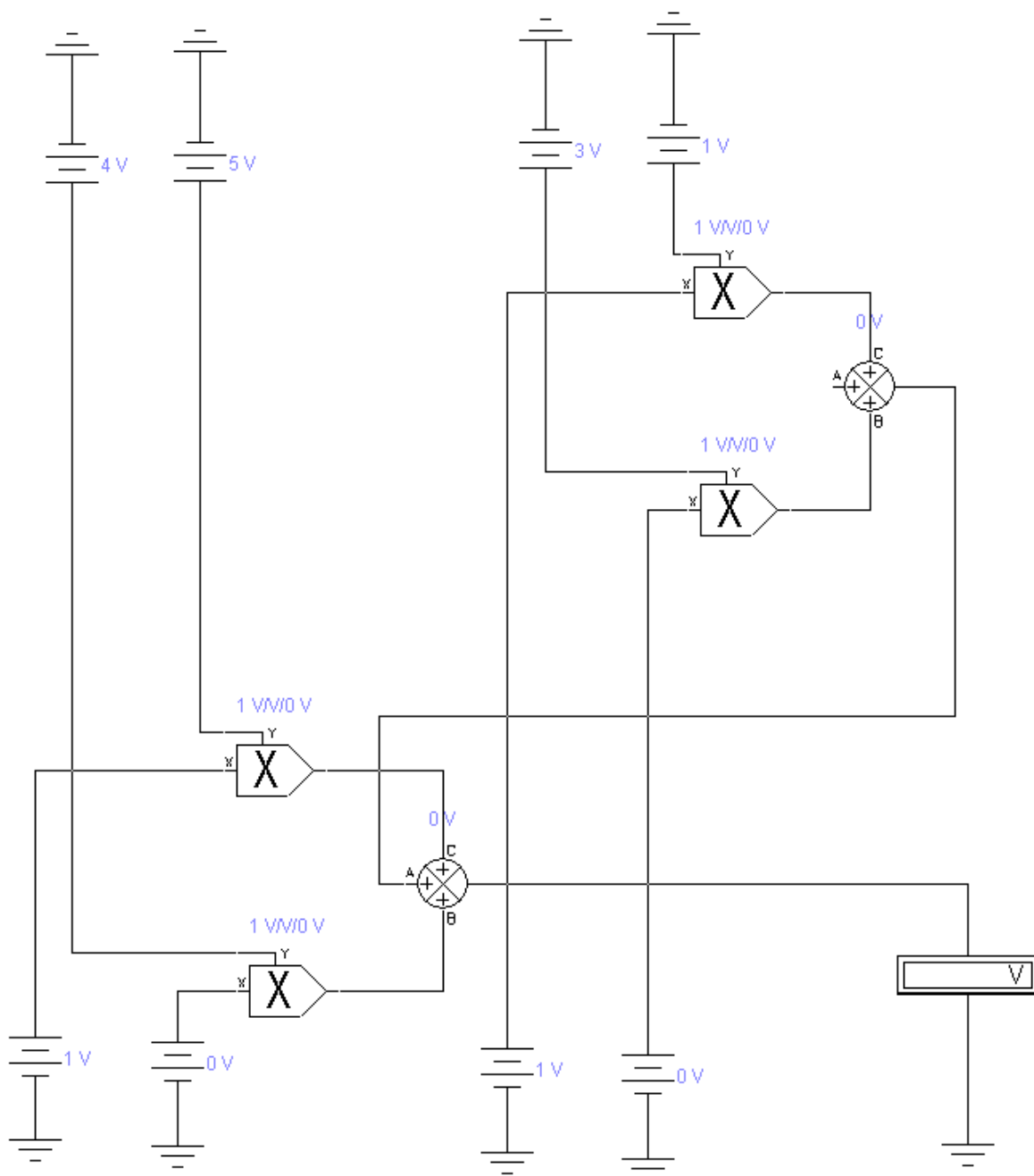
```
1 # Модуль Neuron
2 import numpy as np
3 # Создание класса "Нейрон"
4 class Neuron:
5     def __init__(self, w):
6         self.w = w
7     def y(self, x):
8         s = np.dot(self.w, x) # Суммируем входы
9         return s              # функция активации
10
11 Xi = np.array([1, 0, 0, 1]) # Задание значений входам
12 Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
13 n = Neuron(Wi) # Создание объекта из класса Neuron
14 print ( 'S= ', n.y(Xi)) # Обращение к нейрону
```

Program input

Output

S= 6

[Execution complete with exit code 0]



Итак, сумматор выдаст нам значение $s=6$, а что делать дальше? Как нейрон должен решить, идти на рыбалку или нет? Очевидно, нам нужно как-то преобразовать взвешенную сумму S в итоговое решение. Эту задачу и решает функция активации.

Просто так выводить значение взвешенной суммы пользователю бессмысленно. Нейрон должен как-то обработать ее и сформировать адекватный выходной сигнал. Иными словами, нужно преобразовать взвешенную сумму S в какое-то число, которое и является выходом нейрона (обозначим выходной сигнал переменной Y).

Для разных типов искусственных нейронов используют самые разные функции активации. Обозначим нашу функцию активации как $f(S)$. В данном выражении указание взвешенного сигнала S в скобках означает, что функция активации принимает его как параметр.

Функция активации (activation function) — это такая функция, которая в качестве входного параметра получает взвешенную сумму S как аргумент, а на выходе формирует значение выходного сигнала из нейрона (Y). В общем виде эту функцию можно описать выражением

$$Y = f(S).$$

Рассмотрим некоторые математические функции, которые могут быть использованы в качестве функции активации.

Функция единичного скачка

Общий вид *функции единичного скачка* (или *функции Хевисайда*) приведен на рис. 3.4.

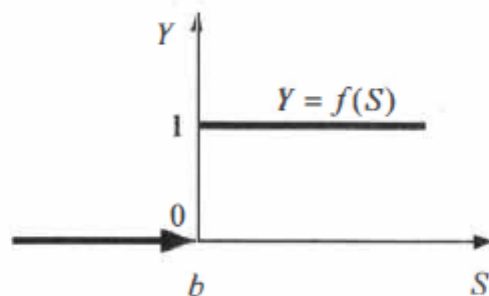


Рис. 3.4. Вид функции единичного скачка

На горизонтальной оси данной функции расположены величины взвешенной суммы S . На вертикальной оси — значения выходного сигнала Y . Это самый простой вид функции активации. При использовании такой функции выход из нейрона Y может получать только два значения — 0 или 1. Какое значение получит выходной сигнал Y на выходе из функции активации, зависит от величины порогового значения b . Если взвешенная сумма S будет больше порога b , то выход нейрона получит значение единица ($Y = 1$). Если сумма S окажется меньше порогового значения b , то выход из нейрона получит значение ноль ($Y = 0$).

Как же можно использовать данную функцию в нашем примере? Предположим, что мы поедem на рыбалку только тогда, когда взвешенная сумма S будет больше

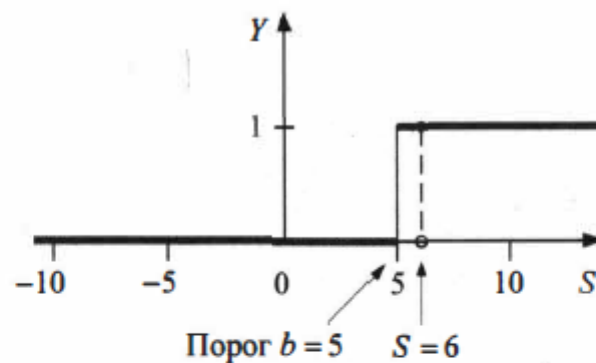


Рис. 3.5. Результаты работы пороговой функции для примера о поездке на море

или равна пороговому значению $b = 5$. В нашем примере взвешенная сумма 6, а это больше 5. Значит, пороговая функция на выходе выдаст значение $Y = 1$ (рис. 3.5).

Проверим это, реализовав работу данной функции активации следующим программным кодом (листинг 3.3).

Листинг 3.3

```
# Модуль onestep
import numpy as np

def onestep(x):
    b = 5
    if x >= b:
        return 1
    else:
        return 0

# Создание класса "Нейрон"
class Neuron:
    def __init__(self, w):
        self.w = w

    def y(self, x): # Сумматор
        s = np.dot(self.w, x) # Суммируем входы
        return onestep(s)     # Функция активации

Xi = np.array([1, 0, 0, 1]) # Задание значений входам
Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
n = Neuron(Wi)              # Создание объекта из класса Neuron
print('Y= ', n.y(Xi))       # Обращение к нейрону
```

В результате работы этой программы получим $Y=1$. Итак, поскольку на выходе искусственный нейрон выдаст значение $Y=1$, то мы принимаем решение — едем ловить рыбу.

Enter a title...

Python



Run

Save

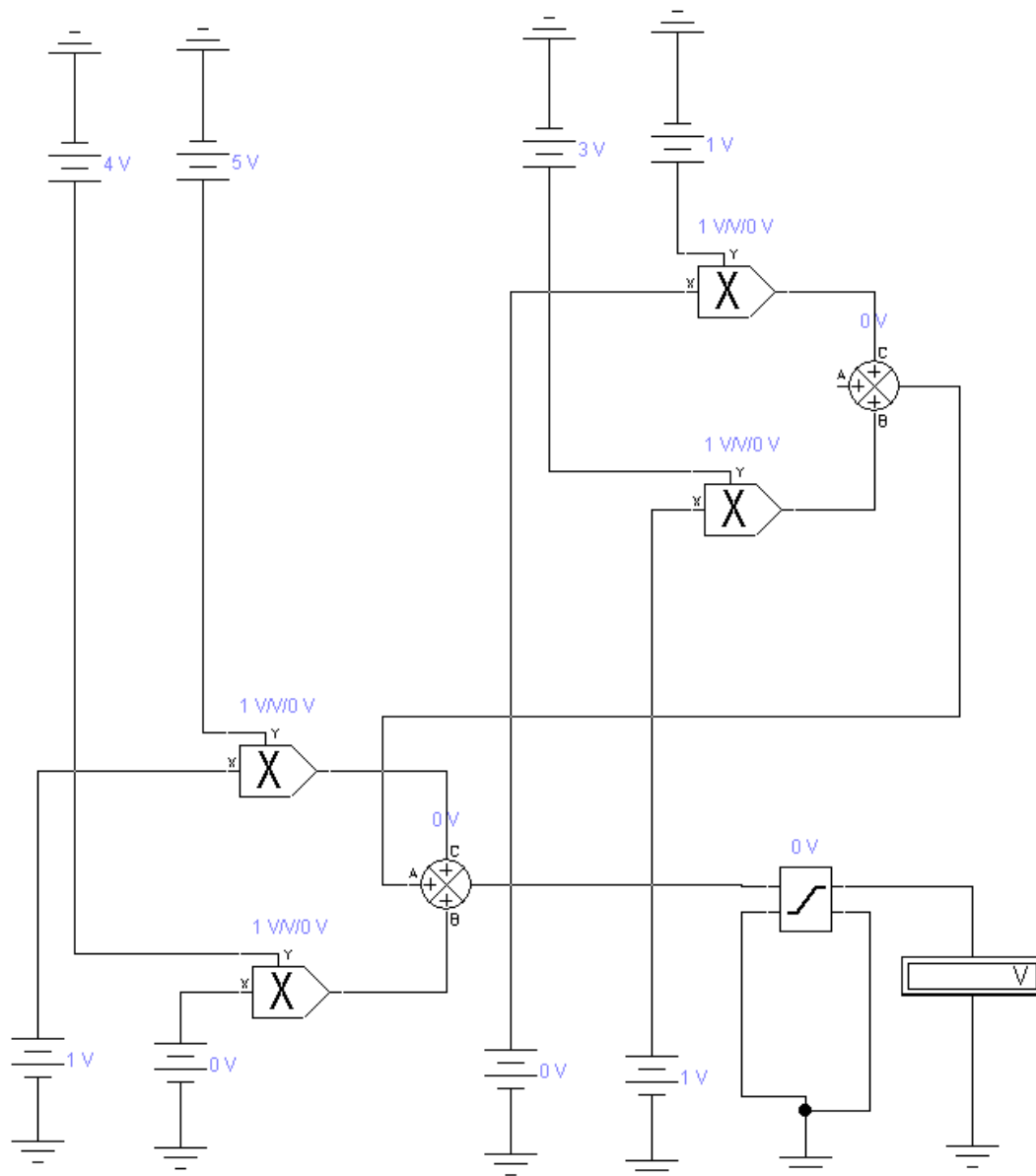
```
1 # Модуль onestep
2 import numpy as np
3 def onestep (x) :
4     b = 5
5     if x >= b:
6         return 1
7     else:
8         return 0
9 # Создание класса "Нейрон"
10 class Neuron:
11     def __init__ (self, w):
12         self.w = w
13     def y(self, x): # Сумматор
14         s = np.dot(self.w, x) # Суммируем входы
15         return onestep(s)     # Функция активации
16 Xi = np.array([1, 0, 0, 1]) # Задание значений входам
17 Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
18 n = Neuron(Wi)              # Создание объекта из класса Neuron
19 print ( 'Y= ', n.y(Xi))     # Обращение к нейрону
```

Program input

Output

Y= 1

[Execution complete with exit code 0]



Теперь рассмотрим работу того же нейрона при условии, что ветер — сильный, атмосферное давление — высокое, на улице пасмурно и температура воды стабильная. В этом случае входные параметры будут иметь следующие значения:

- ☐ ветер (сильный) $x_1 = 0$;
- ☐ атмосферное давление (высокое) $x_2 = 0$;
- ☐ яркость солнца (пасмурно) $x_3 = 1$;
- ☐ перепад температуры воды (нет) $x_4 = 1$.

При поступлении этой информации в сумматор он выдаст следующую итоговую сумму:

$$S = x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 = 0 \cdot 5 + 0 \cdot 4 + 1 \cdot 1 + 1 \cdot 1 = 0 + 0 + 1 + 1 = 2.$$

В этом случае взвешенная сумма $S = 2$, что меньше порогового значения $b = 5$. Значит, на выходе из нейрона получим значение $Y = 0$. В этом случае мы никуда не идем, остаемся дома, поскольку клева не будет. Проверим это с помощью нашей программы (листинг 3.2.3), изменив в ней всего одну строку:

```
Xi = np.array([0, 0, 1, 1]) # Задание значений входам
```

Действительно, на выходе из программы получаем $Y = 0$.

Теперь запишем функцию активации в виде общего математического выражения. В виде составной функции функция единичного скачка будет выглядеть следующим образом:

$$Y = \{0, S < b; 1, S \geq b\}.$$

Читается такая запись следующим образом. Выход нейрона (Y) зависит от взвешенной суммы (S) следующим образом: если S (взвешенная сумма) меньше какого-то порога (b), то Y (выход нейрона) равен 0. А если взвешенная сумма S больше или равна порогу b , то Y равен 1.

Сигмоидальная функция активации

На самом деле существует целое семейство сигмоидальных функций, и некоторые из них применяют в качестве функции активации в искусственных нейронах. Все эти функции обладают полезными свойствами, ради которых их и применяют в нейронных сетях. Эти свойства станут очевидными после того, как мы более детально ознакомимся с графиком этих функций. Итак, наиболее часто используемая в нейронных сетях функция — это *сигмоида*, или *логистическая функция* (рис. 3.6).

График этой функции выглядит достаточно просто, а внешний вид имеет некоторое подобие английской буквы S. А вот так она записывается аналитически:

$$Y = \frac{1}{1 + \exp(-aS)}.$$

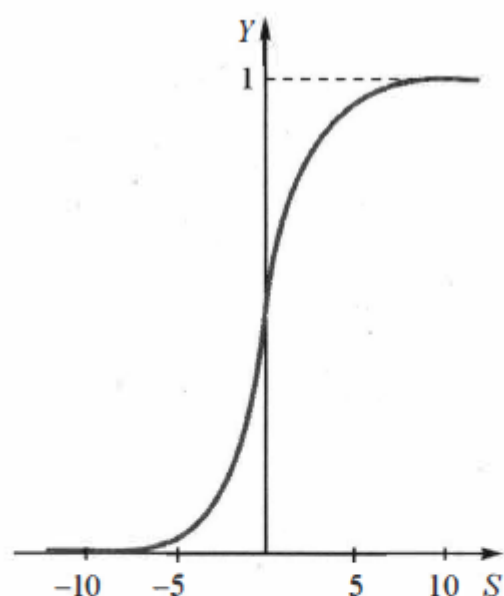


Рис. 3.6. Сигмоида — логистическая функция

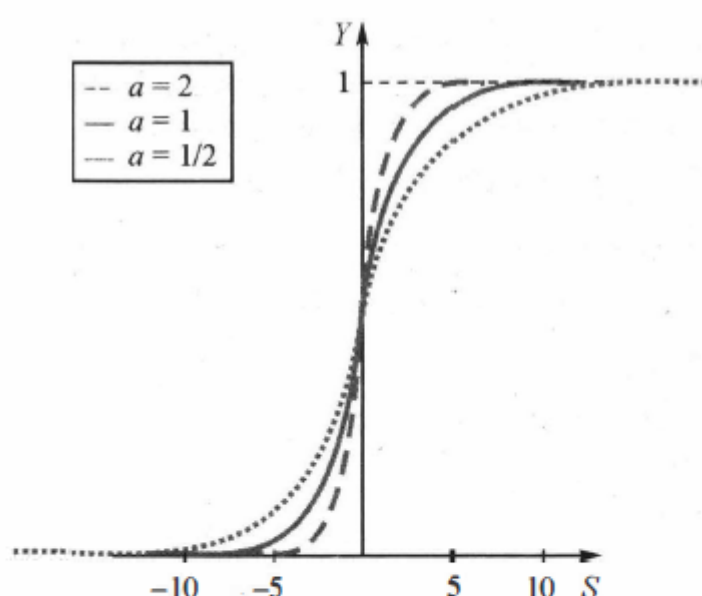


Рис. 3.7. Логистические функции с разными значениями параметра a

Какую роль здесь играет параметр a ? Это некое число, которое характеризует степень крутизны функции. На рис. 3.7 представлены логистические функции с разными значениями параметра a .

Вернемся к нашему искусственному нейрону, который определяет успешность похода на рыбалку. В случае с функцией единичного скачка все было очевидно и однозначно. Мы либо едем на рыбалку $Y=1$, либо нет $Y=0$. При использовании логистической функции мы получаем вероятностный итоговый результат в виде числа между 0 и 1. Причем чем больше взвешенная сумма S , тем ближе выход Y будет к 1 (но никогда не будет точно ей равен). И наоборот, чем меньше взвешенная сумма S , тем выход нейрона Y будет ближе к 0.

Чем ближе значение Y к единице, тем больше вероятность того, что нужно принимать положительное решение. И наоборот, чем ближе значение Y к нулю, тем большая вероятность принятия отрицательного решения. Например, в нашем примере с походом на рыбалку при значении выхода из нейрона $Y=0,8$, скорее всего, пойти ловить рыбу все-таки стоит. Если же значение $Y=0,2$, то это означает, что вам почти наверняка нужно отказаться от такого похода. Для того чтобы однозначно определить итоговое решение, нужно задать величину порогового значения. Например, если задать пороговое значение $b=0,6$, то при рассчитанной величине $Y \geq 0,6$ всегда идем на рыбалку, при $Y < 0,6$ остаемся дома.

Логистическая функция имеет следующие свойства:

- она является "сжимающей" функцией, т. е. вне зависимости от аргумента (взвешенной суммы S) выходной сигнал Y всегда будет в пределах от 0 до 1;
- она более гибкая, чем функция единичного скачка — ее результатом может быть не только 0 и 1, но и любое число между ними;
- во всех точках она имеет производную, и эта производная может быть выражена через эту же функцию.

Именно из-за этих свойств логистическая функция чаще всего используется в качестве функции активации в искусственных нейронах. Немного изменим программный код нашего нейрона и используем в нем сигмоиду в качестве функции активации (листинг 3.4).

Листинг 3.4

```
# Модуль sigmoid
import numpy as np

# функция активации:  $f(x) = 1 / (1 + e^{-x})$ 
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Создание класса "Нейрон"
class Neuron:
    def __init__(self, w):
        self.w = w

    def y(self, x):
        # Сумматор
        s = np.dot(self.w, x) # Суммируем входы
        return sigmoid(s)     # функция активации

Xi = np.array([0, 0, 1, 1]) # Задание значений входам
Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
n = Neuron(Wi)              # Создание объекта из класса Neuron
print('Y= ', n.y(Xi))       # Обращение к нейрону
```

На выходе получим $Y = 0.9975273768433653$, что больше $b = 0,6$, значит, идем на рыбалку.

 myCompiler

English   Recent Login Sign up

Enter a title...

 Python  

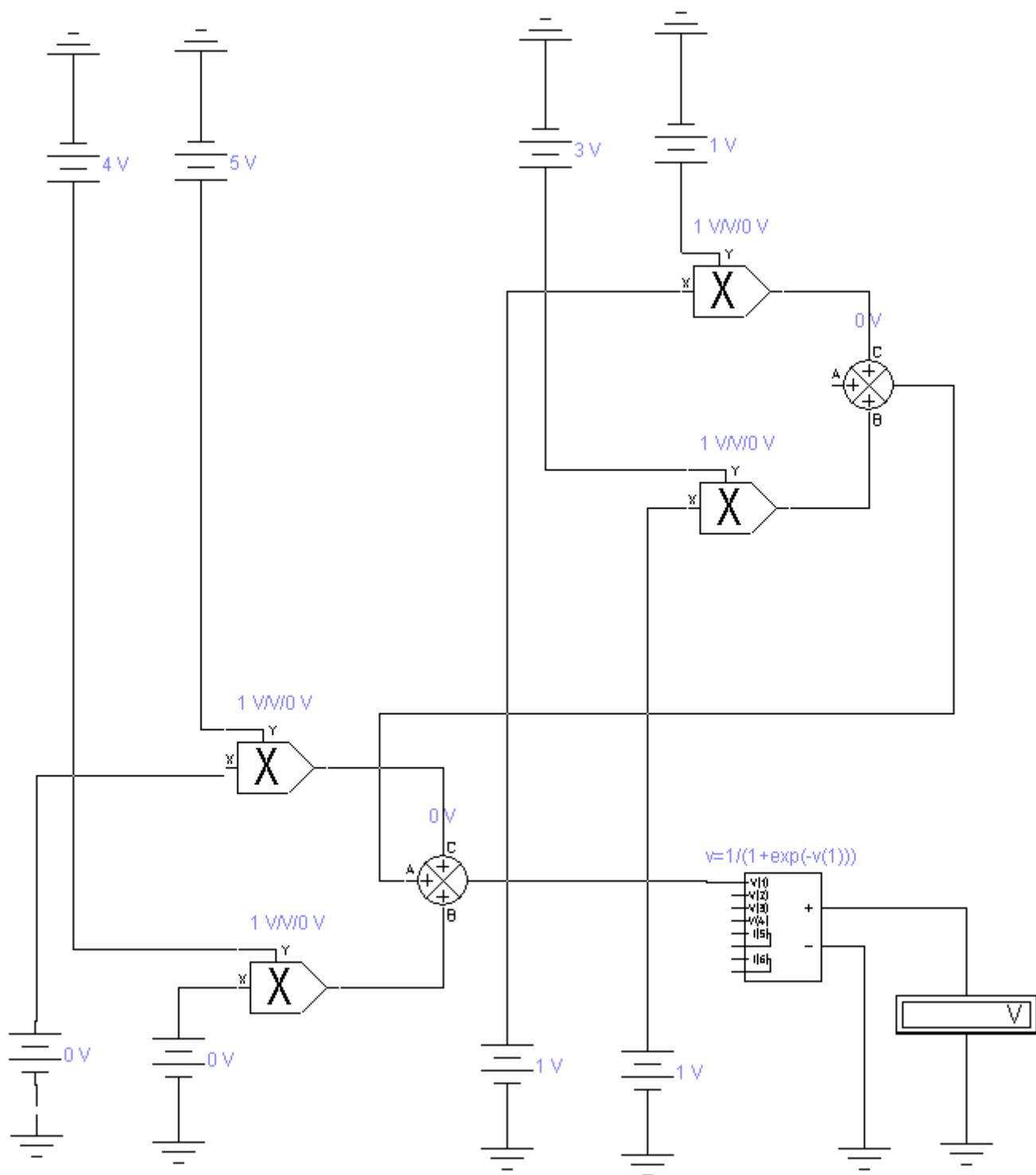
```
1 # Модуль sigmoid
2 import numpy as np
3 # функция активации: f(x) = 1 / (1 + e^(-x))
4 def sigmoid(x):
5     return 1/(1 + np.exp(-x))
6 # Создание класса "Нейрон"
7 class Neuron:
8     def __init__(self, w):
9         self.w = w
10    def y(self, x):
11        # Сумматор
12        s = np.dot(self.w, x) # Суммируем входы
13        return sigmoid(s)     # функция активации
14 Xi = np.array([0, 0, 1, 1]) # Задание значений входам
15 Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
16 n = Neuron(Wi)              # Создание объекта из класса Neuron
17 print('Y= ', n.y(Xi))       # Обращение к нейрону
```

Program input

Output

Y= 0.9820137900379085

[Execution complete with exit code 0]



Изменим входные параметры (самые неблагоприятные погодные условия):

```
Xi = np.array([0, 0, 0, 0]) # Задание значений входам
```

На выходе получим $y = 0.5$, что меньше $b = 0.6$, значит, остаемся дома, клёва не будет.

Enter a title...

Python

3

Run

Save

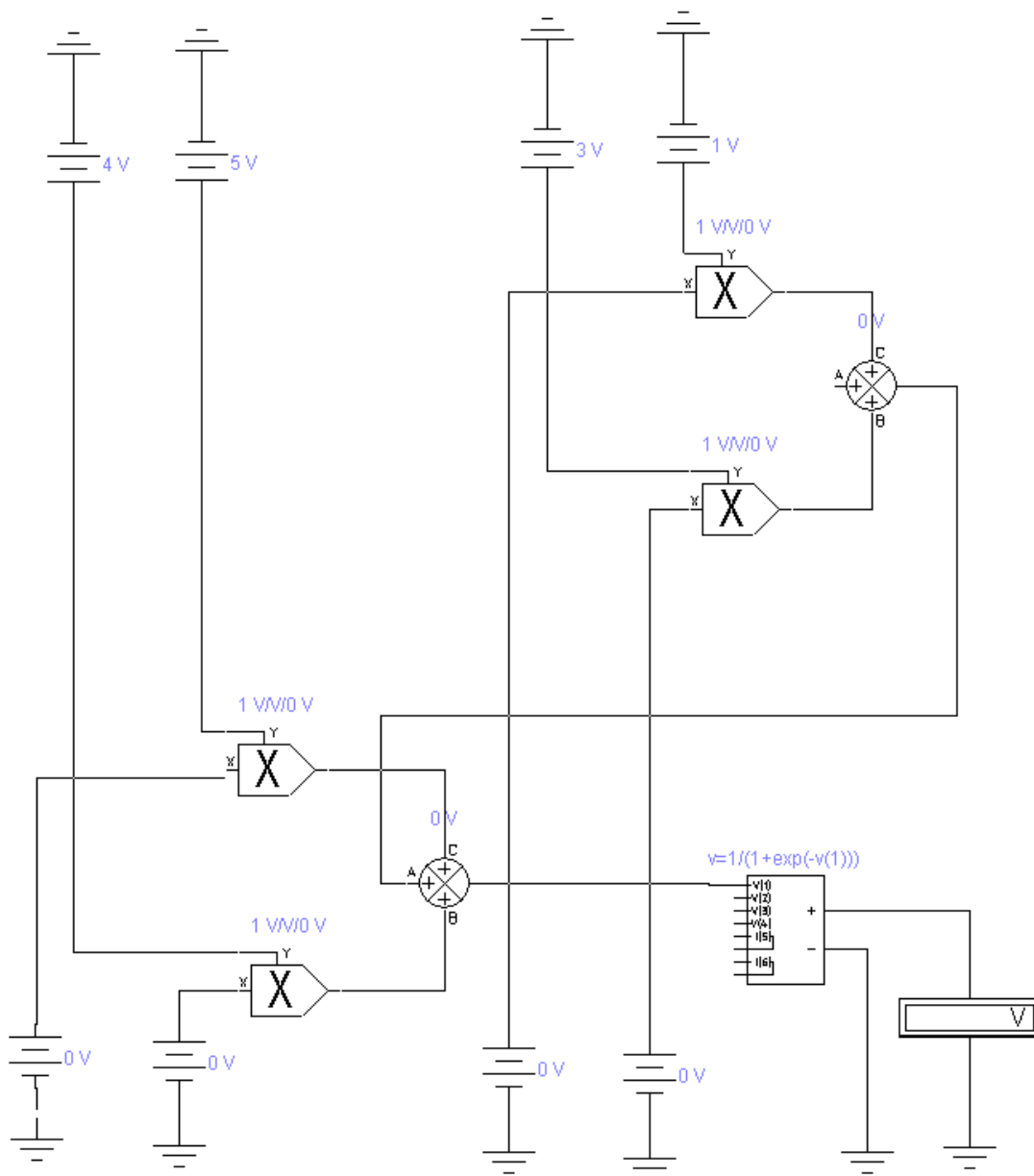
```
1 # Модуль sigmoid
2 import numpy as np
3 # функция активации:  $f(x) = 1 / (1 + e^{-x})$ 
4 def sigmoid(x):
5     return 1/(1 + np.exp(-x))
6 # Создание класса "Нейрон"
7 class Neuron:
8     def __init__(self, w):
9         self.w = w
10    def y(self, x):
11        # Сумматор
12        s = np.dot(self.w, x) # Суммируем входы
13        return sigmoid(s)    # функция активации
14 Xi = np.array([0, 0, 0])    # Задание значений входам
15 Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров
16 n = Neuron(Wi)             # Создание объекта из класса Neuron
17 print('Y= ', n.y(Xi))      # Обращение к нейрону
```

Program input

Output

Y= 0.5

[Execution complete with exit code 0]



Гиперболический тангенс

Однако есть и еще одна сигмоида — *гиперболический тангенс*. Эту функцию чаще применяют биологи для более реалистичной реализации модели нервной клетки. Такая функция позволяет получить на выходе значения разных знаков (от -1 до 1), что может быть полезным для ряда сетей.

Функция записывается следующим образом:

$$Y = \text{th}(S).$$

А график этой функции приведен на рис. 3.8.

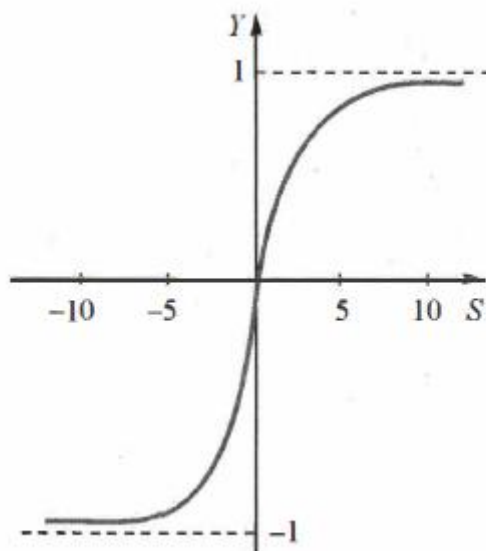


Рис. 3.8. График гиперболического тангенса

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

1. Приведите упрощенную схему искусственного нейрона.
2. Приведите математическую модель искусственного нейрона.
3. Какие существуют активационные функции?
4. Привести программный код на Python реализации сумматоров и искусственных нейронов с различными активационными функциями.
5. Представить решение задачи о походе на рыбалку на основе модели принятия решения с использованием искусственного нейрона.
6. Осуществите реализацию сумматоров и искусственных нейронов моделью аналогового нейровычислителя в Electronics Workbench. Проанализируйте полученные результаты.